

AULA 06

SQL DDL: do modelo às tabelas

Criando o banco no MySQL

Prof. Alysson

14/09/2026

Recapitulando a Aula 05

- **Modelo conceitual:** o negócio, sem tecnologia — entidades, relacionamentos e cardinalidades
- **Modelo lógico:** tabelas, colunas, chaves primárias (PK) e estrangeiras (FK)
- **Modelo físico:** como tudo isso será implementado em um SGBD
- Todo relacionamento **1:N** vira FK no lado N
- Todo relacionamento **N:M** vira tabela associativa

Saímos da Aula 05 com o modelo pronto no papel. Hoje ele vira banco de dados de verdade.

O modelo lógico que vamos implementar

```
TUTOR (id PK, nome, telefone, email UQ)
ANIMAL (id PK, tutor_id FK, nome, especie, raca, data_nascimento)
VETERINARIO (id PK, nome, crmv UQ, especialidade)
CONSULTA (id PK, animal_id FK, veterinario_id FK,
          data_hora, motivo, observacoes)
PROCEDIMENTO (id PK, nome UQ, descricao, valor_referencia)
CONSULTA_PROCEDIMENTO (id PK, consulta_id FK, procedimento_id FK,
                       quantidade, valor_praticado)
```

Clínica **Patas do Cerrado** — seis tabelas.

Como aproveitar esta aula

1. **Leia** estes slides até o fim, sem pressa
2. **Abra o terminal** e execute cada comando na mesma ordem
3. Compare a **saída da sua tela** com a saída mostrada aqui
4. Pare nos quadros **Sua vez** e responda antes de virar o slide
5. Deu erro? **Leia a mensagem inteira** — há slides só sobre isso
6. Ao final: texto base, laboratório e questionário no Google Classroom

Tudo o que você precisa está nos materiais da aula. Dúvida que não se resolver: fórum do Classroom ou WhatsApp da turma.

Nosso mapa de hoje

1. O que é SQL e de onde ele veio
2. As famílias de comandos: DDL, DML, DCL e TCL
3. Abrir o ambiente, subir o MySQL e entrar no cliente
4. `CREATE DATABASE` e `USE`
5. Tipos de dados: escolher com critério
6. `CREATE TABLE` e as restrições
7. Ordem de criação e mensagens de erro
8. Implementar a clínica, tabela por tabela
9. Conferir, testar e corrigir
10. Sua vez: a biblioteca comunitária

Parte 1

O que é SQL

Do papel para o banco

Na Aula 05 escrevemos frases como:

“ `tutor_id` é obrigatório e deve apontar para um tutor existente.”

Hoje escrevemos a mesma regra assim:

```
tutor_id BIGINT UNSIGNED NOT NULL,  
CONSTRAINT fk_animal_tutor FOREIGN KEY (tutor_id)  
REFERENCES tutor (id)
```

SQL é a língua em que o modelo vira estrutura executável.

SQL — *Structured Query Language*

- Linguagem **declarativa**: você descreve **o que** quer, não **como** fazer
- Criada para o **modelo relacional**: tabelas, linhas e colunas
- É a linguagem que praticamente **todos** os bancos relacionais falam
- Serve para **criar**, **alimentar**, **consultar** e **proteger** os dados

Em Java você diz o passo a passo. Em SQL você diz o resultado desejado e o SGBD decide o caminho.

Uma história em quatro nomes

Quem	Quando	O que fez
E. F. Codd	1970	Propôs o modelo relacional (IBM)
Chamberlin e Boyce	1974	Criaram o SEQUEL , que virou SQL
Peter Chen	1976	Criou o Modelo Entidade-Relacionamento
C. J. Date	anos 1980	Consolidou e divulgou a teoria relacional

O DER da Aula 05 e o SQL de hoje nasceram da mesma ideia: dados organizados em relações.

Por que padronizar

- **1986:** primeira norma ANSI (SQL-86)
- **1987:** a ISO adota o padrão — SQL vira norma internacional
- Revisões seguintes: SQL-92, SQL:1999, SQL:2003, SQL:2016, SQL:2023
- Cada SGBD implementa o padrão **e acrescenta** recursos próprios

Dialeto é a versão de SQL de cada fabricante. O núcleo é comum; as bordas mudam.

O mesmo comando, dialetos diferentes

Recurso	MySQL	PostgreSQL	Oracle
Autonumeração	<code>AUTO_INCREMENT</code>	<code>SERIAL</code> / <code>IDENTITY</code>	<code>IDENTITY</code>
Texto variável	<code>VARCHAR(n)</code>	<code>VARCHAR(n)</code>	<code>VARCHAR2(n)</code>
Limitar linhas	<code>LIMIT 10</code>	<code>LIMIT 10</code>	<code>FETCH FIRST 10 ROWS ONLY</code>

O que fazemos aqui vale para qualquer banco relacional; o que muda é o detalhe da escrita.

Nesta disciplina o dialeto é o do **MySQL 8.4**.

Parte 2

As famílias de comandos

DDL, DML, DCL e TCL

Sigla	Nome	Para que serve	Comandos
DDL	<i>Data Definition</i>	Define a estrutura	CREATE , ALTER , DROP , TRUNCATE
DML	<i>Data Manipulation</i>	Mexe nos dados	INSERT , UPDATE , DELETE , SELECT
DCL	<i>Data Control</i>	Controla quem pode o quê	GRANT , REVOKE
TCL	<i>Transaction Control</i>	Confirma ou desfaz	COMMIT , ROLLBACK

Hoje é dia de DDL. A DML começa na Aula 07.

A analogia da casa

- **DDL** — constrói a casa: paredes, cômodos, portas
- **DML** — coloca e retira os móveis
- **DCL** — decide quem recebe a chave de cada porta
- **TCL** — confirma a mudança ou devolve tudo como estava

Móvel nenhum entra antes de existir cômodo. Por isso a DDL vem primeiro.

Um detalhe que assusta depois

No MySQL, comandos DDL têm **confirmação automática** (*commit* implícito).

- Um `INSERT` errado pode ser desfeito com `ROLLBACK`
- Um `DROP TABLE` **não pode**

Antes de apagar qualquer coisa, leia o comando duas vezes.

Regra da disciplina: nada de `DROP DATABASE` em banco que não seja o seu exercício.

Parte 3

Preparando o ambiente

Passo 0 — onde fazer a prática

No navegador, com o ambiente pronto da aula:

1. Acesse <https://labs.iximiuz.com/playgrounds/BD1-AULA06-9e9ee739>
2. Entre na sua conta e clique em **Start Playground**
3. Use a aba **Term** para digitar os comandos — colar: **Ctrl + Shift + V**

A imagem do MySQL 8.4 já vem baixada e os arquivos da aula estão em `~/lab06` .
Nada é instalado no seu computador.

Ou no seu computador, com o Podman da Aula 04. Os comandos são os mesmos nos dois casos.

Subir o MySQL (retomada da Aula 04)

Uma única vez, no seu computador:

```
podman network create rede-bd1  
podman volume create dados-mysql
```

Sempre que for estudar:

```
podman run -d --name mysql-bd1 \  
-e MYSQL_ROOT_PASSWORD=minhasenha \  
-e MYSQL_DATABASE=bd1 \  
-p 3306:3306 -v dados-mysql:/var/lib/mysql \  
--network rede-bd1 mysql:8.4
```

Ou, mais curto, com o `compose.yaml`

O arquivo `compose.yaml` da aula guarda toda a configuração:

```
podman compose up -d      # sobe o servidor
podman compose ps         # confere se está no ar
podman compose logs mysql # lê o log
podman compose down       # desliga (os dados ficam no volume)
```

Se o contêiner já existia, basta `podman start mysql-bd1`.

Os arquivos da aula no ambiente

Quem usa o ambiente do navegador encontra em `~/lab06` :

Arquivo	Para que serve
<code>compose.yaml</code>	sobe o MySQL com <code>podman compose up -d</code>
<code>ddl-clinica.sql</code>	script completo — abra depois de escrever o seu
<code>conferir-clinica.sql</code>	mostra o que você criou

Para conferir o seu trabalho, dentro do cliente MySQL:

```
SOURCE /home/laborant/lab06/conferir-clinica.sql;
```

Entrar no cliente

```
podman exec -it mysql-bd1 mysql -u root -p
```

Digite a senha `minhasenha` . Você chega ao prompt:

```
mysql>
```

Três regras do prompt:

1. Todo comando termina com **ponto e vírgula** `;`
2. Se aparecer `->` , o comando ainda não terminou
3. Para sair: `exit`

Sua vez

Você digitou o comando, apertou Enter, e a tela mostra:

```
->
```

O que aconteceu e o que fazer?

Resposta. Faltou o ponto e vírgula: o MySQL está esperando o resto do comando. Digite `;` e Enter. Se você se perdeu no meio do comando, escreva `\c` e Enter para cancelar e começar de novo.

Parte 4

Criar o banco de dados

CREATE DATABASE

```
CREATE DATABASE IF NOT EXISTS clinica_veterinaria  
  CHARACTER SET utf8mb4  
  COLLATE utf8mb4_0900_ai_ci;
```

- `IF NOT EXISTS` — não dá erro se o banco já existir
- `CHARACTER SET utf8mb4` — aceita **acentos**, ç e emojis
- `COLLATE ..._ai_ci` — comparação **sem** distinguir acento nem maiúscula

Sem `utf8mb4`, “José” pode virar “Jos?” na hora de gravar.

USE: escolher onde trabalhar

```
SHOW DATABASES;  
USE clinica_veterinaria;  
SELECT DATABASE();
```

A última linha responde em qual banco você está:

```
+-----+  
| DATABASE() |  
+-----+  
| clinica_veterinaria |  
+-----+
```

Esqueceu o `USE` ? `ERROR 1046 (3D000): No database selected`

Parte 5

Tipos de dados

Por que o tipo importa

O tipo de dado responde a três perguntas ao mesmo tempo:

1. **O que cabe** naquela coluna (só número? só data?)
2. **Quanto espaço** cada linha vai ocupar
3. **Que erro** o banco deve recusar

O tipo é a **primeira restrição** de uma coluna: escolher `DATE` já impede que alguém grave “ontem de tarde”.

Números inteiros

Tipo	Bytes	Intervalo com sinal	Uso típico
TINYINT	1	-128 a 127	indicadores, idade
SMALLINT	2	-32.768 a 32.767	quantidades pequenas
INT	4	±2,1 bilhões	contadores, códigos
BIGINT	8	±9,2 × 10 ¹⁸	chaves de tabelas grandes

UNSIGNED elimina os negativos e dobra o limite positivo:

SMALLINT UNSIGNED vai de **0 a 65.535**.

Números com casas decimais

Tipo	Como guarda	Use para
DECIMAL(p, s)	exato : p dígitos, s decimais	dinheiro, notas
FLOAT / DOUBLE	aproximado (ponto flutuante)	medidas científicas

```
valor_referencia DECIMAL(10,2) -- até 99.999.999,99
```

Dinheiro nunca em FLOAT. Em ponto flutuante, $0,1 + 0,2$ não dá exatamente $0,3$ — e a clínica fecha o caixa com diferença de centavos.

Texto

Tipo	Limite	Característica
CHAR(n)	255	tamanho fixo , completa com espaços
VARCHAR(n)	65.535 bytes	tamanho variável
TEXT	65.535	texto longo, sem tamanho definido

- CHAR(2) para UF, CHAR(11) para CPF — tamanho sempre igual
- VARCHAR(100) para nome, VARCHAR(150) para e-mail
- TEXT para observações do veterinário

Datas e horas

Tipo	Formato	Use para
DATE	AAAA-MM-DD	nascimento, vencimento
TIME	HH:MM:SS	duração, horário solto
DATETIME	AAAA-MM-DD HH:MM:SS	quando o evento ocorreu
TIMESTAMP	idem, convertido por fuso	auditoria (criado em, alterado em)

No SQL a data se escreve sempre '2026-09-14' — ano, mês, dia. Nunca 14/09/2026 .

Como escolher: três perguntas

1. O conteúdo é **número de contar ou calcular** ou é **código**?
 - CPF e telefone são texto: têm zero à esquerda e não se somam
2. Tem **casas decimais com dinheiro**?
 - `DECIMAL`
3. O tamanho é **sempre o mesmo**?
 - `CHAR` ; senão, `VARCHAR`

Na dúvida entre `VARCHAR(50)` e `VARCHAR(100)` : escolha o que comporta o **maior caso real**, não o caso médio.

Sua vez

Classifique o tipo mais adequado para cada coluna:

1. `crm_v` do veterinário
2. `valor_praticado` de um procedimento
3. `data_hora` da consulta
4. `observacoes` da consulta

Resposta. 1. `VARCHAR(20)` — é código, não número. 2. `DECIMAL(10, 2)` — é dinheiro. 3. `DATETIME` — data e hora do evento. 4. `TEXT` — texto livre, de tamanho imprevisível.

Parte 6

CREATE TABLE

A forma geral

```
CREATE TABLE nome_da_tabela (  
    coluna1 TIPO [restrições da coluna],  
    coluna2 TIPO [restrições da coluna],  
    CONSTRAINT nome_da_restricao TIPO_DA_RESTRICAO (colunas)  
);
```

- Uma linha por coluna, separadas por **vírgula**
- **Sem vírgula** antes do parêntese que fecha
- As restrições podem ficar na coluna **ou** no final, com nome próprio

A primeira tabela

```
CREATE TABLE tutor (  
  id          BIGINT UNSIGNED AUTO_INCREMENT,  
  nome        VARCHAR(100)      NOT NULL,  
  telefone    VARCHAR(20)       NOT NULL,  
  email       VARCHAR(150),  
  CONSTRAINT pk_tutor          PRIMARY KEY (id),  
  CONSTRAINT uq_tutor_email    UNIQUE (email)  
);
```

Query OK, 0 rows affected (0.04 sec)

0 rows affected está certo: DDL cria estrutura, não linhas.

Nomes: a convenção da disciplina

Elemento	Convenção	Exemplo
tabela	singular, minúsculas	<code>animal</code> , <code>consulta_procedimento</code>
coluna	minúsculas com sublinhado	<code>data_nascimento</code>
chave primária	<code>pk_</code> + tabela	<code>pk_tutor</code>
chave estrangeira	<code>fk_</code> + filha + pai	<code>fk_animal_tutor</code>
unicidade	<code>uq_</code> + tabela + coluna	<code>uq_tutor_email</code>
verificação	<code>ck_</code> + tabela + regra	<code>ck_cp_quantidade</code>

Sem acento e sem espaço em nomes de tabela e de coluna.

NULL não é zero nem vazio

NULL significa “**não informado**”.

Situação	Valor guardado
tutor sem e-mail cadastrado	NULL
procedimento gratuito	0.00
observação em branco	' ' (texto vazio)

- NOT NULL → **obrigatório**: a linha não entra sem ele
- Sem NOT NULL → **opcional**: aceita NULL

Cada NOT NULL é uma decisão de negócio, não de sintaxe.

DEFAULT e AUTO_INCREMENT

```
quantidade SMALLINT UNSIGNED NOT NULL DEFAULT 1,  
id          BIGINT UNSIGNED AUTO_INCREMENT,
```

- **DEFAULT** — valor usado quando ninguém informa nada
- **AUTO_INCREMENT** — o MySQL numera sozinho: 1, 2, 3...
 - só em coluna que seja **chave** (normalmente a PK)
 - só **uma** por tabela

É assim que o `id` artificial da Aula 05 se preenche sem trabalho nenhum.

Parte 7

Restrições: as regras do negócio

Cinco restrições, cinco perguntas

Restrição	Pergunta que ela responde
NOT NULL	esse dado pode faltar?
PRIMARY KEY	o que identifica cada linha?
UNIQUE	esse valor pode se repetir?
CHECK	que valores fazem sentido?
FOREIGN KEY	essa linha depende de qual outra?

Escrever restrição é **escrever a regra do negócio dentro do banco**, onde ninguém consegue burlar.

PRIMARY KEY

```
CONSTRAINT pk_tutor PRIMARY KEY (id)
```

- Identifica cada linha de forma **única**
- **Nunca** aceita `NULL` — mesmo sem escrever `NOT NULL`
- Uma única por tabela (pode envolver mais de uma coluna)

Curiosidade útil: o MySQL **ignora** o nome que você dá à PK — ela sempre se chama `PRIMARY` internamente. Escrevemos `pk_tutor` para documentar a intenção e manter o padrão.

UNIQUE

```
CONSTRAINT uq_veterinario_crmv UNIQUE (crmv)
```

- Impede **repetição** de valor na coluna
- Diferente da PK: pode haver **várias** por tabela e ela aceita **NULL**
- Pode combinar colunas:

```
CONSTRAINT uq_consulta_agenda UNIQUE (veterinario_id, data_hora)
```

Essa última linha diz: *o mesmo veterinário não atende duas consultas no mesmo horário.*

id artificial e chave natural

Pergunta que sempre aparece: “se já tem `id`, o CRMV pode repetir?”

Não.

- O `id` é a chave **técnica**: serve às ligações entre tabelas
- O CRMV é a chave **natural**: é a regra do mundo real
- O banco precisa das duas: `PRIMARY KEY (id)` e `UNIQUE (crm)`

Mesmo raciocínio para o ISBN do livro e o e-mail do tutor.

CHECK

```
CONSTRAINT ck_procedimento_valor CHECK (valor_referencia >= 0),  
CONSTRAINT ck_cp_quantidade CHECK (quantidade > 0)
```

- Impõe uma **condição** sobre o valor gravado
- É verificada em toda inserção e em toda alteração
- No MySQL, funciona de verdade a partir da versão **8.0.16**

Valor negativo tentando entrar:

```
ERROR 3819 (HY000): Check constraint 'ck_procedimento_valor' is  
violated.
```

FOREIGN KEY

```
tutor_id BIGINT UNSIGNED NOT NULL,  
CONSTRAINT fk_animal_tutor FOREIGN KEY (tutor_id)  
REFERENCES tutor (id)
```

Três exigências do MySQL:

1. A tabela referenciada (`tutor`) **já precisa existir**
2. Os tipos das duas colunas devem ser **iguais** (`BIGINT UNSIGNED`)
3. A coluna referenciada precisa ser **PK ou UNIQUE**

`NOT NULL` + FK = *todo animal tem tutor, e esse tutor existe.*

O que fazer quando o pai muda

```
CONSTRAINT fk_animal_tutor FOREIGN KEY (tutor_id)
REFERENCES tutor (id)
ON DELETE RESTRICT
ON UPDATE CASCADE
```

Ação	Efeito no filho
RESTRICT	impede apagar o pai que tem filhos
CASCADE	repete a mudança nos filhos
SET NULL	põe NULL no filho (exige coluna opcional)

Padrão da disciplina: RESTRICT no DELETE , CASCADE no UPDATE .

Uma exceção proposital

Na tabela associativa:

```
CONSTRAINT fk_cp_consulta FOREIGN KEY (consulta_id)
REFERENCES consulta (id)
ON DELETE CASCADE
```

Por quê? O item de procedimento **não existe sem a consulta**. Apagada a consulta, os itens dela vão junto.

Já `fk_cp_procedimento` continua `RESTRICT`: o procedimento é catálogo e não some porque alguém apagou uma consulta.

Sua vez

Um colega escreveu:

```
CONSTRAINT fk_consulta_animal FOREIGN KEY (animal_id)
REFERENCES animal (id)
ON DELETE CASCADE
```

Que consequência isso traz para a clínica?

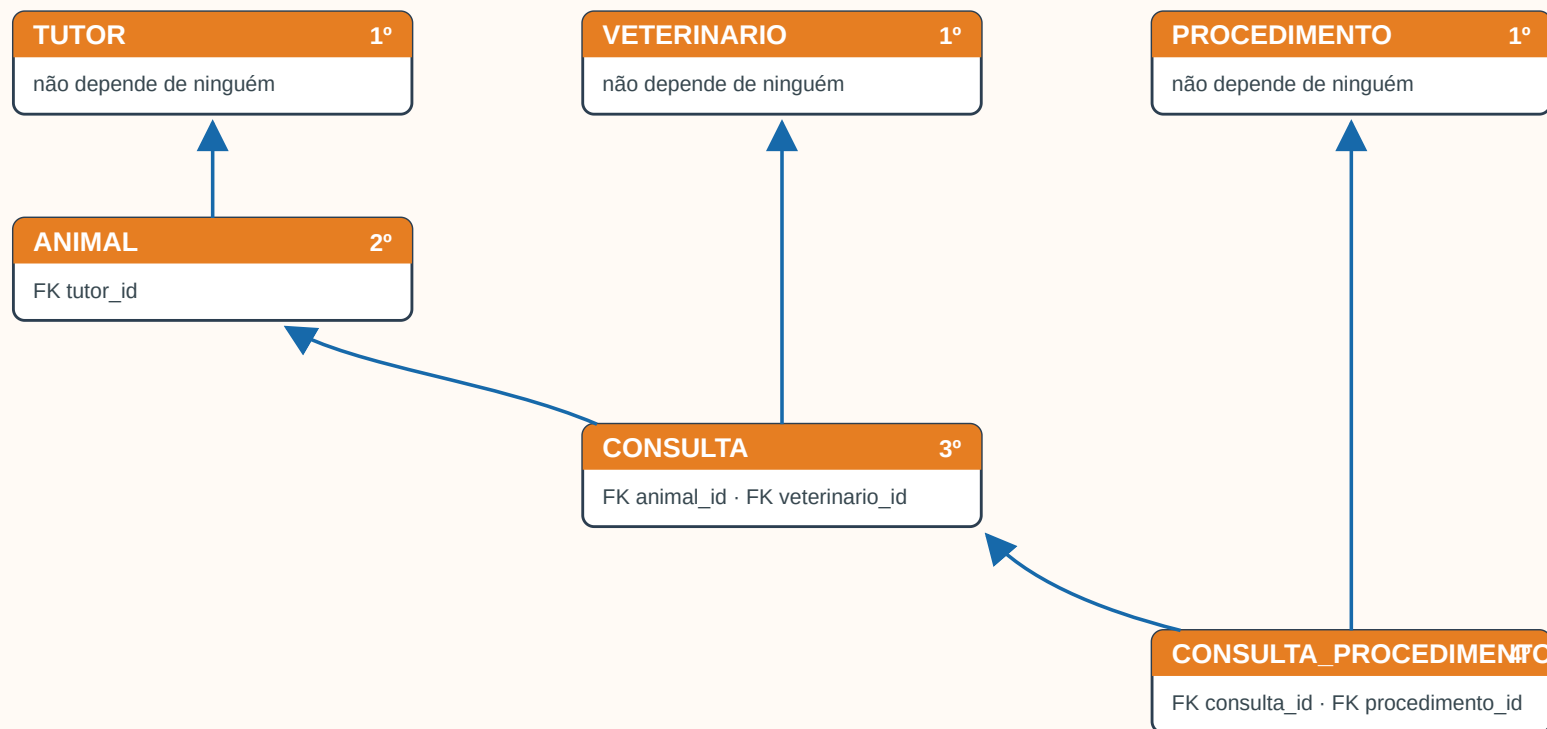
Resposta. Apagar um animal apagaria **todo o histórico de consultas dele**, silenciosamente. Em histórico clínico isso é grave: o correto é `RESTRICT`, que impede apagar o animal enquanto houver consultas registradas.

Parte 8

Ordem de criação e erros

Quem depende de quem

ORDEM DE CRIAÇÃO DAS TABELAS — CLÍNICA PATAS DO CERRADO



A seta aponta para a tabela que precisa existir antes · para apagar, siga o caminho inverso

A regra da ordem

Crie primeiro a tabela que não depende de ninguém.

1. `tutor`, `veterinario`, `procedimento` — não têm FK
2. `animal` — depende de `tutor`
3. `consulta` — depende de `animal` e `veterinario`
4. `consulta_procedimento` — depende de `consulta` e `procedimento`

Para **apagar**, o caminho é o inverso: comece pelas tabelas de cima.

Erro: tabela referenciada não existe

Criando `animal` antes de `tutor` :

```
ERROR 1824 (HY000): Failed to open the referenced table 'tutor'
```

Tradução: “não achei a tabela que sua FK aponta”.

Correção: crie `tutor` primeiro e repita o comando.

Em outras versões a mesma situação aparece como `ERROR 1215: Cannot add foreign key constraint`.

Erro: tipos incompatíveis

`tutor.id` é `BIGINT UNSIGNED`, mas alguém declarou `tutor_id INT`:

```
ERROR 3780 (HY000): Referencing column 'tutor_id' and referenced  
column 'id' in foreign key constraint 'fk_animal_tutor' are  
incompatible.
```

Correção: a coluna da FK precisa ter **exatamente** o mesmo tipo da coluna referenciada — inclusive o `UNSIGNED`.

Erros de digitação e de repetição

```
ERROR 1064 (42000): You have an error in your SQL syntax... near  
' ) '
```

Quase sempre: **vírgula sobrando** antes do parêntese final, parêntese não fechado ou palavra escrita errada. Olhe o trecho citado **e a linha anterior**.

```
ERROR 1050 (42S01): Table 'tutor' already exists
```

A tabela já foi criada. Confira com `SHOW TABLES;` .

Como ler qualquer erro do MySQL

```
ERROR 1824 (HY000): Failed to open the referenced table 'tutor'
```

_____/ _____/ _____/

número o que houve objeto citado

1. **Número** — identifica o tipo do problema
2. **Texto** — diz o que o servidor não conseguiu fazer
3. **Objeto** — mostra onde olhar primeiro

Erro não é castigo: é o banco recusando dado errado. Leia, corrija, execute de novo.

Parte 9

Implementando a clínica

Passo 1 — o banco

```
CREATE DATABASE IF NOT EXISTS clinica_veterinaria  
  CHARACTER SET utf8mb4  
  COLLATE utf8mb4_0900_ai_ci;  
  
USE clinica_veterinaria;
```

Confira antes de seguir:

```
SELECT DATABASE();
```

Precisa responder `clinica_veterinaria` .

Passo 2 — tutor e veterinario

```
CREATE TABLE tutor (  
  id          BIGINT UNSIGNED AUTO_INCREMENT,  
  nome        VARCHAR(100)      NOT NULL,  
  telefone    VARCHAR(20)       NOT NULL,  
  email        VARCHAR(150),  
  CONSTRAINT pk_tutor          PRIMARY KEY (id),  
  CONSTRAINT uq_tutor_email    UNIQUE (email)  
);  
  
CREATE TABLE veterinario (  
  id          BIGINT UNSIGNED AUTO_INCREMENT,  
  nome        VARCHAR(100)      NOT NULL,  
  crmv        VARCHAR(20)       NOT NULL,  
  especialidade VARCHAR(80),  
  CONSTRAINT pk_veterinario    PRIMARY KEY (id),  
  CONSTRAINT uq_veterinario_crmv UNIQUE (crmv)  
);
```

Passo 3 — procedimento

```
CREATE TABLE procedimento (  
  id          BIGINT UNSIGNED AUTO_INCREMENT,  
  nome        VARCHAR(100)      NOT NULL,  
  descricao   VARCHAR(255),  
  valor_referencia DECIMAL(10,2) NOT NULL,  
  CONSTRAINT pk_procedimento PRIMARY KEY (id),  
  CONSTRAINT uq_procedimento_nome UNIQUE (nome),  
  CONSTRAINT ck_procedimento_valor CHECK (valor_referencia >= 0)  
);
```

Três regras do negócio, três restrições: identidade, nome que não repete e valor que não fica negativo.

Passo 4 — animal

```
CREATE TABLE animal (  
  id          BIGINT UNSIGNED AUTO_INCREMENT,  
  tutor_id    BIGINT UNSIGNED NOT NULL,  
  nome        VARCHAR(80)      NOT NULL,  
  especie     VARCHAR(40)      NOT NULL,  
  raca        VARCHAR(60),  
  data_nascimento DATE,  
  CONSTRAINT pk_animal PRIMARY KEY (id),  
  CONSTRAINT fk_animal_tutor FOREIGN KEY (tutor_id)  
    REFERENCES tutor (id)  
    ON DELETE RESTRICT ON UPDATE CASCADE  
);
```

`raca` e `data_nascimento` ficam opcionais: animal resgatado pode chegar sem essa informação.

Passo 5 — consulta

```
CREATE TABLE consulta (  
  id          BIGINT UNSIGNED AUTO_INCREMENT,  
  animal_id   BIGINT UNSIGNED NOT NULL,  
  veterinario_id BIGINT UNSIGNED NOT NULL,  
  data_hora   DATETIME        NOT NULL,  
  motivo      VARCHAR(255)     NOT NULL,  
  observacoes TEXT,  
  CONSTRAINT pk_consulta PRIMARY KEY (id),  
  CONSTRAINT fk_consulta_animal FOREIGN KEY (animal_id)  
    REFERENCES animal (id) ON DELETE RESTRICT ON UPDATE CASCADE,  
  CONSTRAINT fk_consulta_veterinario FOREIGN KEY (veterinario_id)  
    REFERENCES veterinario (id) ON DELETE RESTRICT ON UPDATE CASCADE,  
  CONSTRAINT uq_consulta_agenda UNIQUE (veterinario_id, data_hora)  
);
```

Passo 6 — consulta_procedimento

```
CREATE TABLE consulta_procedimento (  
  id                BIGINT UNSIGNED AUTO_INCREMENT,  
  consulta_id       BIGINT UNSIGNED NOT NULL,  
  procedimento_id   BIGINT UNSIGNED NOT NULL,  
  quantidade        SMALLINT UNSIGNED NOT NULL DEFAULT 1,  
  valor_praticado   DECIMAL(10,2) NOT NULL,  
  CONSTRAINT pk_consulta_procedimento PRIMARY KEY (id),  
  CONSTRAINT fk_cp_consulta FOREIGN KEY (consulta_id)  
    REFERENCES consulta (id) ON DELETE CASCADE ON UPDATE CASCADE,  
  CONSTRAINT fk_cp_procedimento FOREIGN KEY (procedimento_id)  
    REFERENCES procedimento (id) ON DELETE RESTRICT ON UPDATE CASCADE,  
  CONSTRAINT uq_consulta_procedimento UNIQUE (consulta_id, procedimento_id),  
  CONSTRAINT ck_cp_quantidade CHECK (quantidade > 0),  
  CONSTRAINT ck_cp_valor CHECK (valor_praticado >= 0)  
);
```

Por que `valor_praticado` existe

O catálogo guarda `valor_referencia`. A linha da consulta guarda `valor_praticado`.

- O preço de tabela **muda** com o tempo
- O que foi cobrado **naquele atendimento** não pode mudar depois

Guardar o valor praticado é registrar o **fato histórico**. Esse cuidado aparece em qualquer sistema com preço: venda, contrato, matrícula.

Parte 10

Conferir, testar e corrigir

Conferir o que foi criado

```
SHOW TABLES;
```

```
+-----+
| Tables_in_clinica_veterinaria |
+-----+
| animal                         |
| consulta                      |
| consulta_procedimento         |
| procedimento                  |
| tutor                         |
| veterinario                   |
+-----+
6 rows in set (0.00 sec)
```

Seis tabelas. Faltou alguma? Volte ao passo correspondente.

Conferir uma tabela

```
DESCRIBE animal;
```

Field	Type	Null	Key	Default	Extra
id	bigint unsigned	NO	PRI	NULL	auto_increment
tutor_id	bigint unsigned	NO	MUL	NULL	
nome	varchar(80)	NO		NULL	
especie	varchar(40)	NO		NULL	
raca	varchar(60)	YES		NULL	
data_nascimento	date	YES		NULL	

PRI = chave primária · **MUL** = coluna de FK · **Null: YES** = opcional

Ver a tabela como o banco a guardou

```
SHOW CREATE TABLE animal\G
```

Mostra o comando **completo**, com tudo que o MySQL acrescentou: motor de armazenamento, conjunto de caracteres e as restrições nomeadas.

```
SELECT table_name, constraint_name, constraint_type  
FROM information_schema.table_constraints  
WHERE table_schema = 'clinica_veterinaria';
```

O `\G` troca a tabela por uma lista vertical — bem mais legível no celular e em telas estreitas.

Testar de propósito

A melhor forma de acreditar nas restrições é **tentar quebrá-las**:

```
INSERT INTO animal (tutor_id, nome, especie)
VALUES (999, 'Rex', 'Canino');
```

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign
key constraint fails
```

O tutor 999 não existe — e o banco **recusou** o dado órfão. A estrutura está funcionando.

Corrigir sem recomeçar: ALTER TABLE

```
-- acrescentar coluna
ALTER TABLE tutor ADD COLUMN cidade VARCHAR(60);

-- mudar o tipo ou a obrigatoriedade
ALTER TABLE animal MODIFY COLUMN raca VARCHAR(80) NOT NULL;

-- renomear coluna
ALTER TABLE tutor RENAME COLUMN telefone TO celular;

-- acrescentar restrição
ALTER TABLE tutor ADD CONSTRAINT uq_tutor_celular UNIQUE (celular);

-- remover restrição
ALTER TABLE tutor DROP CONSTRAINT uq_tutor_celular;
```

Errou uma coluna? **Não apague o banco.** Ajuste com ALTER TABLE . São exemplos de sintaxe: se executar, desfça depois para a estrutura continuar igual à

DROP e TRUNCATE

```
DROP TABLE IF EXISTS consulta_procedimento; -- apaga estrutura e dados
TRUNCATE TABLE consulta_procedimento;      -- apaga só os dados
```

- DROP desmonta a tabela inteira; **não há ROLLBACK**
- TRUNCATE esvazia e reinicia a contagem do AUTO_INCREMENT
- Tabela referenciada por FK **não é apagada**:

```
ERROR 3730 (HY000): Cannot drop table 'tutor' referenced by a
foreign key constraint
```

Sua vez

Você precisa recomeçar o exercício do zero, apagando as seis tabelas. Por onde começa?

Resposta. Pelas tabelas que **dependem** das outras, na ordem inversa da criação: `consulta_procedimento`, `consulta`, `animal` e, por último, `tutor`, `veterinario` e `procedimento`. Começando por `tutor`, o MySQL recusa com o erro 3730.

Parte 11

Sua vez: biblioteca comunitária

O modelo da biblioteca comunitária

Você chegou a ele na atividade final da Aula 05:

```
LEITOR (id PK, nome, telefone)
LIVRO (id PK, titulo, isbn UQ)
EXEMPLAR (id PK, livro_id FK, codigo_tombo UQ)
EMPRESTIMO (id PK, leitor_id FK, exemplar_id FK,
            data_retirada, data_devolucao_prevista,
            data_devolucao_real)
```

Tarefa: escrever o script DDL completo desse modelo, no banco `biblioteca` .

O que o seu script precisa ter

1. `CREATE DATABASE biblioteca` com `utf8mb4`
2. As quatro tabelas na **ordem certa** de dependência
3. `id` `BIGINT UNSIGNED AUTO_INCREMENT` e PK em todas
4. `NOT NULL` no que é obrigatório: nome, título, tombo, datas de retirada e de previsão
5. `UNIQUE` em `isbn` e em `codigo_tombo`
6. FK em `livro_id`, `leitor_id` e `exemplar_id`
7. `data_devolucao_real` **opcional** — o empréstimo em aberto ainda não tem

Duas decisões para justificar

- `data_devolucao_real` aceita `NULL`. Por quê? Porque `NULL` aqui significa “*ainda não devolveu*” — é exatamente o que diferencia o empréstimo em aberto do encerrado.
- `ON DELETE` do empréstimo. Apagar um leitor que tem empréstimo registrado? `RESTRICT`: o histórico da biblioteca não pode sumir por causa de um cadastro.

Confira o seu script no laboratório da aula e no gabarito comentado publicado após o prazo.

Checklist do seu DDL

- [] o banco foi criado com `utf8mb4`
- [] toda tabela tem `id` e `PRIMARY KEY`
- [] as tabelas foram criadas na ordem das dependências
- [] cada FK aponta para uma coluna PK/UNIQUE do mesmo tipo
- [] cada coluna obrigatória tem `NOT NULL`
- [] cada chave natural tem `UNIQUE`
- [] cada regra de valor virou `CHECK`
- [] `SHOW TABLES` mostra o número esperado de tabelas
- [] `DESCRIBE` de cada tabela confere com o modelo lógico

O que aprendemos hoje

- SQL nasceu do modelo relacional e é **padronizado** pela ANSI/ISO
- A **DDL** define estrutura; DML, DCL e TCL cuidam do resto
- O **tipo de dado** é a primeira restrição de cada coluna
- Cada **restrição** representa uma regra do negócio
- A **ordem de criação** segue as dependências de FK
- **Mensagem de erro** é diagnóstico: número, texto e objeto
- `SHOW` , `DESCRIBE` e `ALTER` permitem conferir e corrigir

Atividade e próximos passos

Para esta semana

1. Refazer o laboratório até que as seis tabelas sejam criadas sem erro
2. Escrever o DDL da biblioteca comunitária
3. Responder o questionário da Aula 06 no Google Classroom

Aula 07 — 21/09

Com o banco criado, começa a DML: `INSERT`, `UPDATE` e o primeiro `SELECT`.

Guarde o seu script: ele é o embrião do `init.sql` exigido na A2. No ambiente do navegador, escolha **Stop** ao sair — assim você retoma a máquina como deixou.

Fim